

Knowledge and Language 2025/2026

literature Review

Miguel Castela, uc2022212972
DEI, Universidade de Coimbra
uc2022212972@student.uc.pt

Miguel Martins, uc2022213951
DEI, Universidade de Coimbra
uc2022213951@student.uc.pt

I. ABSTRACT

This literature review explores the integration of Knowledge Graphs (KGs) and Retrieval-Augmented Generation (RAG) techniques for domain-specific question answering. The challenge we are trying to tackle is building a system that can reliably and accurately answer complex, domain-specific questions about Portuguese cuisine and other recipes using the food.com dataset [1]. We will use RecipeRAG [1], a KG-driven recipe generation framework, as reference for the methodologies to build a food chatbot. Definitions and concepts are supported by the class materials on semantic web framework (RDF, OWL, SPARQL), establishing the link between symbolic and knowledge graph representation.

II. CONNECTIONS WITH THE TOPICS COVERED BY THE COURSE

Large Language Models (LLMs) demonstrate strong generalization but fail in domain-specific reasoning due to three main limitations: (1) lack of contextual understanding, (2) incomplete factual knowledge, and (3) weak recall of precise facts [2]. This motivates the integration of symbolic knowledge (RDF/OWL-based graphs) with neural inference (LLMs and RAG) to ensure both reasoning and factual grounding. RDF can be serialized in formats such as N-TRIPLES, XML, JSON-LD, or Turtle, ensuring interoperability across systems [3]. Vocabularies define terms, relationships and ontologies define classes, hierarchies, and property restrictions. OWL (Web Ontology Language) can be used with RDF to express richer logical constraints, such as domain/range restrictions and class relations [3].

This symbolic foundation aligns with the three “knowledge-aware” dimensions of KG-augmented NLP methods outlined in [2]:

- **KG-Augmented Retrieval:** The use of structured queries (SPARQL) or textualized triples to retrieve relevant information for LLMs, comparable to RAG pipelines [4].
- **KG-Augmented Reasoning and Controlled Generation:** Incorporating graph reasoning steps (symbolic inference or logical validation) within the generation loop.
- **Knowledge-Aware Training:** Using KG-derived data to fine-tune LLMs, injecting factual consistency.
- **Knowledge-Aware Validation:** Employing SPARQL or rule-based checks to verify the factual correctness of generated answers.

This emphasizes the distinction between symbolic reasoning (FOL-based) and neural reasoning, emphasizing that symbolic models provide interpretability and factual precision, while neural models capture language variability and contextual generation.

III. RDFS/OWL KNOWLEDGE GRAPHS FOR REPRESENTING KNOWLEDGE FROM DATASETS

In RDF, knowledge is represented through triples — for example:

`<Bacalhau_a_Brás> <hasIngredient> <Egg>`

Each triple can be enriched through RDFS with class and property hierarchies and constraints. In [5] RecipeKG is constructed as a large-scale recipe knowledge graph combining ingredients, cooking techniques, and regions, and enriching it with semantic annotations. Enabling structured querying and semantic interoperability through URIs and ontologies.

IV. KNOWLEDGE RETRIEVAL VIA NLP

Knowledge retrieval connects natural language input to structured or embedded KG representations. SPARQL is the declarative query language for RDF graphs [3], enabling semantic retrieval using triples and filters. SPARQL queries operate over RDF graphs, and their construction can be made in 3 different ways:

- The user using the Chatbot need to input the SPARQL query directly or use a form-based interface that generates the query based on user selections.
- LLMs can be fine-tuned or prompted to generate SPARQL queries from natural language questions, using few-shot examples or templates to guide the generation.
- NLP tools like semantic parsers can convert natural language into SPARQL queries by identifying entities, intent, relations, restrictions and query structure.

Natural language interfaces require NLP-based query interpretation, extracting entities and intents (via tools like spaCy or BERT [6]) and mapping them to KG entities (URIs). In the system, BERT and spaCy are used together to understand user queries about recipes. BERT is fine-tuned to identify the user’s intent—such as finding a recipe, retrieving ingredients, or saving a recipe—which enables the system to correctly

interpret what the user wants even when questions are phrased in different ways. BERT’s bidirectional transformer architecture provides deep contextual understanding, improving both the identification of user intents and entity disambiguation in complex queries. spaCy [7] is used to extract entities and linguistic features from user queries. Its efficient, rule and model-based pipeline enables fast and accurate identification of nouns, verbs, entities, and syntactic relations, helping map natural language input to knowledge graph entities and improving the system’s overall understanding of user intents.

KGE-based retrieval [5] significantly outperforms text-based retrieval when handling multiple user constraints. As noted: “Performance declined as the number of criteria increased... Nevertheless, the results underscore the effectiveness of KGEs in supporting multi-criteria recipe retrieval, while simultaneously revealing the limitations of purely text-based embeddings.” Symbolic KGs are inherently sparse, so in RecipeRAG Knowledge Graph Embeddings (KGEs) are employed to map entities and relations into continuous vector spaces. Foundational models [5] include TransE, RotatE, and QuatE, representing relations as translations, rotations, and quaternion transformations respectively.

V. RETRIEVAL-AUGMENTED GENERATION (RAG)

RAG, first formalized by [4], is a hybrid model that retrieves external documents as the input sequence x to retrieve text documents z and use them as additional context when generating the target sequence y . This approach separates retrieval from generation, improving factuality and reducing hallucinations. While the original RAG used text documents, later adaptations like the mentioned RecipeRAG [5] replace text retrieval with KG-based retrieval.

Lewis et al. (2020) introduced the Retrieval-Augmented Generation (RAG) model, a hybrid architecture that combines parametric and non-parametric memory systems to improve knowledge-intensive natural language generation. The parametric memory is represented by a pre-trained seq2seq model [8] (a transformer base model [9]), specifically BART, which stores implicit knowledge within its parameters. However, such knowledge is static, cannot be easily updated or interpreted, and may lead to factual inaccuracies or “hallucinations.” To address these issues, RAG incorporates a non-parametric memory — a dense vector index of Wikipedia built using the Dense Passage Retriever (DPR). Each passage is encoded as a dense vector, allowing the model to retrieve the top- k most relevant documents for a given query through Maximum Inner Product Search (MIPS):

$$p_{\eta}(z | x) \propto \exp(d(z)^{\top} q(x)) \quad (1)$$

where The retrieval component of RAG, denoted $p_{\eta}(z | x)$, represents the probability of retrieving a document z given a query x . Each document z is encoded into a dense vector representation $d(z)$, while the query x is encoded into a corresponding dense vector $q(x)$. The retriever computes the similarity between the query and each document using the

Maximum Inner Product Search (MIPS) criterion, defined as

$$p_{\eta}(z | x) \propto \exp(d(z)^{\top} q(x)),$$

and retrieves the documents that maximize this inner product:

$$\arg \max_z d(z)^{\top} q(x).$$

This operation identifies the documents whose embeddings are most similar to the query embedding, ensuring that the most relevant passages are selected for the generator to use during response generation. During generation, RAG first uses DPR to retrieve relevant documents and then conditions the [10] generator on both the input and the retrieved passages. The final output is produced by marginalizing over all retrieved documents, effectively combining symbolic retrieval with neural generation. Lewis et al. proposed two variants: RAG-Sequence, which uses the same document for the whole output, and RAG-Token, which allows different documents to inform each generated token. This means that for each token y_i being generated, the model computes a weighted sum over the probabilities of that token conditioned on each of the top- k retrieved documents. This architecture enables RAG to generate more factual and interpretable text, while maintaining the fluency of neural models. Unlike traditional language models that rely solely on fixed parametric memory, RAG can update its knowledge dynamically by replacing the document index, provide evidence for its outputs through retrieved passages, and reduce hallucinations, achieving superior performance on knowledge-intensive tasks such as open-domain question answering and fact verification.

Two model variants were proposed: *RAG-Sequence*, which conditions generation on a single retrieved document for the entire sequence, and *RAG-Token*, which allows token-level retrieval, providing finer factual grounding. Their experiments demonstrated state-of-the-art performance across knowledge-intensive tasks such as Natural Questions, TriviaQA, and FEVER, outperforming both parametric-only (e.g., T5, BART) and purely extractive retrieval models (e.g., DPR, REALM). Notably, RAG models also produced more specific, diverse, and factually correct generations, showing reduced hallucination and enabling knowledge updates simply by replacing the underlying index.

RecipeRAG [5] builds upon this foundation, substituting text retrieval with KG-based retrieval and applying it to domain-specific recipe generation. The authors state:

“We chose RotatE, the strongest-performing KGE model, to bootstrap the recipe generation process. The system retrieves the top-ranking recipes based on aggregated relevance scores, selects the top 5, and feeds them—together with user criteria—into the generator (DeepSeek-R1-0528).”

This pipeline combines the retrieval mechanism of [4] with structured KG enrichment, turning symbolic knowledge into context for generative models.

RAG thus operationalizes the Knowledge-Aware Inference paradigm [2], where external structured knowledge is used to enhance reasoning and factual correctness.

VI. CONCLUSION AND FUTURE WORK

The reviewed literature converges on the same insight: combining symbolic reasoning (RDF, OWL, SPARQL) and neural reasoning (KGE, RAG, LLMs) yields systems that are both factual and context-aware. Empirically, RecipeRAG [5] and RAG [4] demonstrate that KG-driven retrieval enhances generation quality, while the taxonomy of [2] clarifies how such systems reduce hallucination through Knowledge-Aware Inference, Learning, and Validation.

REFERENCES

- [1] S. Li, “Food.com recipes and interactions dataset,” <https://www.kaggle.com/datasets/shuyangli94/food-com-recipes-and-user-interactions>, 2020, accessed: 2024-06-10.
- [2] A. Agrawal, R. Patel, and M. Singh, “Can knowledge graphs reduce hallucinations in llms? a survey,” in *Proceedings of NAACL 2024*, 2024.
- [3] “Semantic web technologies — rdf, rdfs, owl, sparql, and knowledge representation slides,” 2024, course materials, PowerPoint slides.
- [4] P. Lewis, E. Perez, A. Piktus, V. Karpukhin, N. Goyal, A. Kulshreshtha, W. Murdock, A. Fan, M. Li, and F. Petroni, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*. Facebook AI Research, 2020.
- [5] T. Loesch, Y. Zhang, and A. Kumar, “Reciperag: A knowledge graph-driven approach to personalized recipe retrieval and generation,” in *Proceedings of RAGE-KG 2025*, 2025.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2019, submitted 11 Oct 2018 (v1); last revised 24 May 2019 (v2). [Online]. Available: <https://doi.org/10.48550/arXiv.1810.04805>
- [7] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, “spacy: Industrial-strength natural language processing in python,” <https://zenodo.org/record/1215370>, 2020, software release.
- [8] I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” *arXiv preprint arXiv:1409.3215*, 2014. [Online]. Available: <https://arxiv.org/abs/1409.3215>
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *arXiv preprint arXiv:1706.03762*, 2017, submitted 12 Jun 2017 (v1); last revised 2 Aug 2023 (v7). [Online]. Available: <https://doi.org/10.48550/arXiv.1706.03762>
- [10] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, “Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” *arXiv preprint arXiv:1910.13461*, 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1910.13461>